Agent Phantom Recovery

07-trd.md

Technical Requirements Document (TRD)

Part 2

Agent, Memory, Reasoning, Tooling, Verification & Security Requirements

---

9. Agent Requirements

9.1 Agent Philosophy

The system shall operate as a long-horizon engineering agent rather than a conversational assistant.

The primary objective of the agent is:

```
Investigate
↓
Reason
↓
Execute
↓
Verify
↓
Deliver
```

The system must prioritize task completion over response generation.

9.2 Agent Operating Model

The platform shall follow a continuous execution cycle:

```
Goal
↓
Plan
```

```
↓
Select Tool
↓
Execute Action
↓
Observe Result
↓
Update State
↓
Replan
↓
Continue
```

This cycle may repeat dozens or hundreds of times during a complex task.

9.3 Long-Horizon Execution

The system shall support:

- Extended workflows
- Multi-stage investigations
- Long-running repository analysis
- Multi-hour execution sessions
- Multi-tool coordination

The architecture must assume that important tasks cannot be solved in a single reasoning pass.

9.4 Autonomous Task Progression

The agent shall continue working toward a goal without requiring confirmation after every action.

Human intervention should only be required when:

- permissions are needed
- safety boundaries are reached
- critical ambiguity exists
- execution cannot continue

9.5 Goal Persistence

The agent shall maintain awareness of:

- primary goal
- current sub-goal
- completed steps
- failed approaches

- pending actions

Goal persistence must survive:

- tool switches
- context summarization
- memory compression
- execution restarts

9.6 Adaptive Planning

The agent shall be capable of:

- abandoning invalid hypotheses
- updating plans
- changing strategies
- selecting different tools
- recovering from failed approaches

The system must not rigidly follow an outdated plan.

9.7 Agent State Machine

Supported states:

```
Idle
↓
Intake
↓
Planning
↓
Evidence Gathering
↓
Analysis
↓
Execution
↓
Verification
↓
Delivery
↓
Completed
```

Additional states:

```
Paused
Blocked
Waiting For Input
Failed
Recovery Mode
```

---

10. Memory Requirements

10.1 Memory Philosophy

Memory exists to prevent repeated rediscovery.

The system should remember:

- facts
- observations
- failures
- successful approaches
- project context

Memory should improve future decision quality.

10.2 Memory Layers

The platform shall implement four memory categories.

Working Memory

Short-lived context used during active execution.

Stores:

- active goal
- current plan
- recent observations
- current files
- active tool state

Expected lifetime:

- current task execution

Session Memory

Stores information generated during a session.

Examples:

- commands executed
- files inspected
- tests executed
- findings generated

Expected lifetime:

- current session

Project Memory

Stores repository-specific knowledge.

Examples:

- architecture understanding
- dependency relationships
- important modules
- recurring issues

Expected lifetime:

- project lifespan

Experience Memory

Stores strategy-outcome relationships.

Examples:

```
Attempt:
Run Integration Tests First

Outcome:
Slow

Result:
Avoid For Small Changes
```

```
Attempt:
Inspect Authentication Middleware
```

```
Outcome:
Root Cause Found

Result:
Useful Pattern
```

Expected lifetime:

- cross-project learning

10.3 Memory Requirements

The memory system shall support:

- retrieval
- storage
- updating
- summarization
- ranking
- pruning

Memory must remain searchable.

10.4 Context Compression

When context windows become large:

The system shall:

- summarize older content
- preserve critical facts
- remove redundancy
- maintain traceability

The system must avoid losing key information during compression.

10.5 Memory Retrieval

Retrieved memories shall be ranked using:

- relevance
- recency
- importance
- confidence

Low-value memories should not dominate retrieval.

---

11. Reasoning Requirements

11.1 Reasoning Philosophy

The system shall not immediately act on the first hypothesis.

Reasoning should follow:

```
Question
↓
Analysis
↓
Verification
↓
Conclusion
```

11.2 Multi-Step Reasoning

The system shall support:

- decomposition
- dependency analysis
- causal reasoning
- architecture reasoning
- failure analysis

The platform should be capable of reasoning across large workflows.

11.3 Root Cause Reasoning

The system shall distinguish between:

Symptom

Example:

```
Test Failed
```

Root Cause

Example:

```
Authentication middleware returns
invalid state after token refresh.
```

The platform should prioritize root-cause discovery.

11.4 Evidence-Based Reasoning

Every major conclusion shall be supported by evidence.

Examples:

- code references
- logs
- test results
- browser observations
- dependency analysis

Unsupported claims must be flagged.

11.5 Architectural Reasoning

The platform shall understand:

- system architecture
- module interactions
- dependencies
- side effects
- compatibility constraints

This is required for safe patch generation.

11.6 Side-Effect Analysis

Before applying a fix, the system shall evaluate:

- dependency impact
- performance impact
- API compatibility
- behavioral changes
- regression risk

11.7 Self-Critique

The system shall perform internal review.

Workflow:

```
Generate
↓
Critique
↓
Improve
↓
Verify
```

The goal is to reduce incorrect conclusions before verification.

---

12. Tooling Requirements

12.1 Tool Philosophy

Tools are first-class components.

The platform should use tools to collect evidence rather than relying solely on model reasoning.

12.2 Supported Tool Categories

Terminal

Capabilities:

- command execution
- builds
- tests
- package management
- environment inspection

Browser

Capabilities:

- website rendering
- navigation
- observation
- state inspection
- screenshot capture

The browser must be embedded within the workspace.

File System

Capabilities:

- read
- write
- modify
- delete
- search

Search

Capabilities:

- documentation lookup
- repository search
- indexed retrieval
- knowledge discovery

Code Execution

Capabilities:

- scripting
- automation
- data processing
- analysis

Git / GitHub

Capabilities:

- clone
- branch creation
- commit generation
- diff generation
- pull request preparation

12.3 Tool Selection

Tool selection must be:

- evidence-driven
- goal-driven
- context-aware

Tools should not be invoked unnecessarily.

12.4 Tool Traceability

Every tool action shall record:

- timestamp
- input
- output
- duration
- result

This supports debugging and auditing.

12.5 Tool Failure Recovery

The platform shall support:

- retries
- fallback tools
- alternative strategies
- failure logging

Tool failure should not automatically terminate execution.

---

13. Verification Requirements

13.1 Verification Philosophy

Verification is a mandatory stage.

Nothing should be considered complete until verified.

13.2 Verification Pipeline

```
Finding
↓
Evidence Review
↓
Patch Review
↓
Testing
↓
```

```
Verifier Review
↓
Acceptance
```

13.3 Verifier Responsibilities

The verifier shall:

- challenge findings
- challenge assumptions
- inspect patches
- detect regressions
- validate reasoning

The verifier's role is skepticism.

13.4 Evidence Verification

Claims must be checked against:

- logs
- code
- browser observations
- test outputs
- repository evidence

13.5 Patch Verification

Before approval:

- tests must pass
- patch must be reviewed
- regression risk must be evaluated

13.6 Verification Output

Verification results shall include:

- pass/fail status
- supporting evidence
- unresolved concerns
- confidence assessment

---

14. Security Requirements

14.1 Security Philosophy

The platform is security-aware but not security-exclusive.

Security capabilities exist to support:

- defensive analysis
- remediation
- recovery
- validation

The primary focus remains engineering outcomes.

14.2 Authorization Requirement

Security investigations shall only occur within authorized environments.

The system shall assume authorization is required.

14.3 Security Investigation Workflow

```
Evidence Collection
↓
Candidate Identification
↓
Analysis
↓
Verification
↓
Remediation Design
↓
Validation
↓
Documentation
```

14.4 Security Evidence Standards

Findings shall require:

- supporting code
- supporting behavior

- supporting logs
- reproducible reasoning

Speculation alone is insufficient.

14.5 Vulnerability Correlation

The system should identify relationships between findings.

Examples:

```
Issue A
+
Issue B
+
Issue C
↓
Higher-Risk Condition
```

The goal is understanding system risk, not producing exploitation workflows.

14.6 Security Fix Requirements

Security fixes shall prioritize:

1. Correctness
2. Compatibility
3. Maintainability
4. Performance
5. Regression Prevention

14.7 Secure Execution

The platform shall support:

- permission boundaries
- execution isolation
- auditability
- action logging

Security-sensitive actions should remain traceable.

---

15. Workspace Requirements

15.1 Browser Workspace

The platform shall provide an embedded browser surface capable of rendering real websites.

15.2 Source Code Workspace

The platform shall provide:

- file navigation
- code viewing
- patch review
- diff visualization

15.3 Terminal Workspace

The terminal shall remain docked and visible as a primary execution surface.

Capabilities:

- command execution
- test output
- build logs
- execution traces

15.4 Agent Console

The workspace shall expose:

- current goal
- current plan
- current action
- recent observations
- verification results

The user should always understand what the agent is doing.

End of TRD Part 2

Next Section:

- Infrastructure Requirements

- Scalability Requirements
- Reliability Requirements
- Observability Requirements
- Data Requirements
- API Requirements
- Service Architecture Requirements
- Storage Requirements
- Event Architecture
- Integration Requirements